

SQL Injection

Complete Theory Notes

Interview Preparation & Pentesting Reference Guide

What's Inside

- Core Concepts & How Applications Talk to Databases
- Database Accounts, Parsers & Why Injection Works
- Prepared Statements & Placeholders
- Injection Techniques: Error-Based, UNION-Based, Blind (Boolean & Time-Based)
- information_schema & Metadata Enumeration
- Authentication Bypass & SQL Comments
- Full Exploitation Workflow
- Defenses: Prepared Statements, Validation, Least Privilege
- Key Interview Definitions

1. What Is SQL Injection?

SQL Injection (SQLi) is a vulnerability where user input becomes part of an SQL query because the application directly inserts user input into the query instead of treating it as data.

Example — Application builds:

```
SELECT * FROM users WHERE username='$username';
```

Normal input:

```
admin
```

Query becomes:

```
SELECT * FROM users WHERE username='admin';
```

Malicious input:

```
admin' OR '1'='1
```

Query becomes:

```
SELECT * FROM users
WHERE username='admin'
OR '1'='1';
```

Since **'1'='1'** is always TRUE, the WHERE condition becomes TRUE, so multiple rows (or all rows) may be returned.

Main idea: SQL Injection is NOT about attacking the database directly. It is about tricking the application into sending a different SQL query to the database.

2. How a Web Application Talks to MySQL

```
User
↓
Web Application
↓
MySQL Database
```

The process:

- 1 User submits input.
- 2 The web application creates an SQL query.
- 3 The web application sends that query to MySQL.

- 4 MySQL parses and executes the query.
- 5 Results return to the application.
- 6 Application sends the response back to the user.

Important: the web application CREATES and SENDS the query. MySQL EXECUTES the query.

3. Database Account

The web application has its own account inside MySQL.

```
Username: webshop
Password: *****
```

When the application connects to MySQL, it authenticates using this account. Every SQL query is executed using the permissions of this account.

The database account is NOT software. It is simply an identity (credentials) used to authenticate with the database.

4. Why SQL Injection Is Dangerous

The attacker never logs into MySQL directly. Instead:

```
Attacker
  ↓
Changes user input
  ↓
Application creates malicious SQL
  ↓
Application sends SQL
  ↓
MySQL executes it using the application's database account.
```

The attacker only gets the permissions of that database account.

5. What Is a Parser?

A parser is the component inside MySQL that understands SQL.

```
SELECT username
FROM users
WHERE id = 5;
```

The parser identifies:

Token	Type
SELECT	SQL keyword
username	column
FROM	keyword
users	table
WHERE	keyword
id	column
5	integer

Without a parser, MySQL would only see random text. It would never understand SQL commands.

6. Why SQL Injection Works

Suppose the application creates:

```
SELECT * FROM users
WHERE username='admin' OR '1'='1';
```

MySQL receives the entire query. The parser reads everything. Since OR is valid SQL syntax, it becomes part of the query.

The parser has no idea which part came from the developer or the user — everything is treated as SQL.

7. Prepared Statements

Prepared statements stop SQL Injection. Instead of building:

```
SELECT * FROM users
WHERE username='admin';
```

the application sends:

```
SELECT * FROM users
WHERE username = ?
```

The ? is a placeholder. MySQL now:

- 1 Parses the SQL.
- 2 Checks syntax.
- 3 Creates an execution plan.

The SQL structure is now FIXED. Later the application sends:

```
admin' OR '1'='1
```

This is NOT sent as SQL — it is sent only as the parameter value. MySQL binds the value to the placeholder. The parser does NOT parse SQL again. Instead, MySQL simply compares:

```
username == "admin' OR '1'='1"
```

Since that username probably doesn't exist, 0 rows are returned. The OR never becomes SQL — it stays part of the string.

8. Placeholders

A placeholder does NOT mean “insert SQL here.” It means “insert ONE value here.”

That value can be:

- String
- Integer
- Float
- Boolean
- Date

Whatever is sent becomes ONE parameter value. It is never interpreted as SQL.

9. Error-Based SQL Injection

Purpose: confirm that SQL Injection exists.

```
?id=1'
```

The quote breaks the SQL query. If SQL errors appear, the application is probably vulnerable.

10. UNION-Based SQL Injection

Purpose: return our own data together with application data.

UNION combines two SELECT statements. This lets us retrieve database information.

11. Why Find Column Count?

UNION only works when both SELECT statements return the same number of columns. We test:

```
UNION SELECT 1  
  
UNION SELECT 1,2  
  
UNION SELECT 1,2,3
```

Eventually one succeeds. Now we know how many columns exist.

12. Reflected Columns

Some columns appear on the webpage. Example:

```
UNION SELECT 1,2,3
```

Website displays:

```
2  
3
```

Now we know columns 2 and 3 are reflected. These are where we place functions such as `database()`, `group_concat()`, `user()`, `version()`, etc.

13. database()

Returns the current database name. Example:

```
shop
```

Knowing the database name allows us to enumerate its contents.

14. What Is Metadata?

Metadata = information about other data. Database metadata describes:

- Database names
- Table names
- Column names
- Data types
- Primary keys
- Constraints

Metadata DOES NOT contain usernames, passwords, emails, or actual table records. Those are stored inside the actual tables.

15. information_schema

information_schema is a special database that exposes metadata. Think of it as the database directory. It tells us: databases, tables, columns.

```
Database
shop
↓
Tables
users, orders, products
↓
Columns
username, password, email
```

After discovering these, we query the ACTUAL tables. *information_schema* never stores the actual user data — it only tells us WHERE the data is.

16. Why Do Pentesters Use information_schema?

When we first discover SQL Injection, we know nothing — not the database names, table names, or column names. Without information_schema, we would have to guess. Instead we ask the database itself.

Database → Tables → Columns

Once we know these, we query the real tables to extract data.

17. Why Does MySQL Need Metadata?

Suppose we run:

```
INSERT INTO users(username,password)
```

Before inserting, MySQL must know:

- Does users exist?
- Does username exist?
- Is username VARCHAR?
- Is id auto increment?
- Which constraints exist?

Without metadata, MySQL cannot answer these questions, and a relational database cannot function.

Important: MySQL depends on metadata. `information_schema` is simply one interface for viewing that metadata.

18. Tables vs. Data

Table:	users
Columns:	id, username, password
Actual data:	1 admin p4ssword 2 john hello123

Metadata describes the table. The table stores the data.

19. group_concat()

Normally SQL returns separate rows:

```
admin  
john  
alice
```

`group_concat()` combines them into a single value:

```
admin,john,alice
```

Useful for displaying many rows in a single output.

20. Boolean-Based Blind SQLi

The application never displays data. Instead it returns TRUE or FALSE. We ask questions like:

```
Does database start with "s"? → TRUE  
Does database start with "sq"? → TRUE
```

Continue until the value is discovered.

21. Time-Based Blind SQLi

Instead of observing TRUE/FALSE, we observe delay or no delay. Example:

```
SLEEP(5)
```

If the response delays, the condition is TRUE. Otherwise, FALSE.

22. Authentication Bypass

Original query:

```
WHERE username='admin'  
AND password='test'
```

Payload:

```
admin' --
```

Everything after -- becomes a comment. The password check disappears; only the username is checked.

23. SQL Comments

Common comment markers:

```
--  
#
```

Purpose: ignore everything after our payload.

24. SQL Injection Workflow

- 1 Find SQL Injection
- 2 Confirm vulnerability
- 3 Find column count
- 4 Find reflected columns
- 5 Find database
- 6 Use information_schema
- 7 Find tables
- 8 Find columns
- 9 Query real tables
- 10 Extract data
- 11 Use sqlmap after manual confirmation if appropriate

25. Defenses

1. Prepared Statements

Best defense. SQL and user input are kept separate. The parser finishes before user data is supplied. User data becomes only a parameter — no SQL Injection.

2. Input Validation

Checks whether input is acceptable (e.g. a Product ID should contain only numbers). It reduces attack surface, but is NOT enough by itself — prepared statements are still required.

3. Least Privilege

The application's database account should receive only the permissions it actually needs.

Permission	Granted?
SELECT	✓
INSERT	✓
UPDATE	✓
DROP TABLE	✗
CREATE USER	✗

If SQL Injection happens, the attacker is limited by those permissions.

26. Key Interview Definitions

Metadata

Information that describes other data.

information_schema

A special database that exposes database metadata such as database names, table names, columns, data types, and constraints.

Database Account

A user account used by the application to authenticate to the database. All SQL queries execute using that account's permissions.

Parser

The component that converts SQL text into a structure MySQL understands and can execute.

Prepared Statement

A mechanism where SQL is parsed before user input is supplied. User input is bound as parameters rather than becoming SQL code.

Least Privilege

Giving an account only the permissions it actually needs so that the impact of a compromise is minimized.